



Describing a Programming Language

2

A programming language is an artificial formalism in which algorithms can be expressed. For all its artificiality, though, this formalism remains a *language*. Its study can make good use of the many concepts and tools developed in the last century in linguistics (which studies both natural and artificial languages). Without going into great detail, this chapter poses the problem of what it means to “give” (define) a programming language and which tools can be used in this undertaking.

2.1 Levels of Description

In a study that has now become a classic in linguistics, Morris [1] studied the various levels at which a description of a language can occur. He identified three major areas: *grammar*, *semantics* and *pragmatics*.

Grammar is that part of the description of the language which answers the question: which phrases are correct? Once the alphabet of a language has been defined as a first step (in the case of natural language, for example, the Latin alphabet of 22 or 26 letters, the Cyrillic alphabet, etc.), the *lexical* component, which uses this alphabet, identifies the sequence of symbols constituting the *words* (or *tokens*) of the language defined. When alphabet and words have been defined, the *syntax* describes which sequences of words constitute legal phrases. Syntax is therefore a relation between signs. Between all possible sequences of words (over a given alphabet), the syntax chooses a subset of sequences to form phrases of the language proper.¹

¹ In linguistics, obviously, things are more complicated. In addition to the lexical and the syntactic levels, there is also a morphological level which is distinct from the two previous ones. At the morphological level, the different forms assumed by words (or phrases) as a function of their

Semantics is that part of the description of the language which seeks to answer the question “what does a correct phrase mean?” Semantics, therefore, attributes a *significance* to every correct phrase. In the case of natural languages, the process of attribution of meaning can be very complex; in the case of artificial languages the situation is simpler. It is not difficult to assume, in this case, that semantics is a relation between signs (correct sentences) and meanings (autonomous entities existing independently of the signs that are used to describe them). For example, the meaning of a certain program could be the mathematical function computed by that program. Or, it could be the trace of the changes the program induces on the state of an abstract machine. The semantic description of that language will be constructed using techniques allowing us, when given a program, to fix the function the program computes; or to determine the changes on the abstract machine.

It is at the third level that the principal actor makes its appearance on the scene, the person who uses a certain language. *Pragmatics* is that part of a language description which asks itself “how do we use a meaningful sentence?” Sentences with the same meaning can be used in different ways by different users. Different linguistic contexts can require the use of different sentences; some are more elegant, some are antiquated, or more dialect-based than others. Pragmatics is interested in the intersubjective, social aspects of a language—understanding these aspects is no less important than knowing the syntax and semantics.

In the case of programming languages, we can add a fourth level to the three classical ones: the *implementation* level. Given that the languages that interest us are procedural languages (that is, languages whose correct phrases specify actions), it remains for us to describe “how to execute a correct sentence, in such a way that we respect the semantics”. A knowledge of the semantics is, in general, enough for the language user, but the software designer (and more importantly the language designer) is also interested in the process with which operative phrases implement the actions they specify. It is precisely this which is described by the language implementation.

We can give a fairly rudimentary example which we hope will serve our purposes. Let us consider the natural language used to express recipes in cooking. The syntax determines the correct sentences with which a recipe is expressed. The semantics is about explaining “what is” a recipe, independent of its (specific) execution. Pragmatics studies how a cook (“that cook”) interprets the various sentences of the recipe. In the end, the implementation describes the way (where, and with what ingredients) the kitchen recipe transforms into the dish that the semantics prescribes.

In the next sections, we analyse the role performed by the four levels when they are applied to programming languages.

grammatical function are defined. For example in the lexicon (that is, the dictionary, the thesaurus of the language in question), we find the word “bear” with the associated lexical value given by the image of the animal that everybody knows. At the morphological level, on the other hand, the word is convertible into the root “bear” and the morpheme “-s” which signals the plural. The natural language’s phonetic aspects are also present but they are of no interest to us here.

2.2 Grammar and Syntax

We have already said that the grammar of a language first establishes the alphabet and lexicon. Then by means of a syntax, it defines those sequences of symbols corresponding to well-formed phrases and sentences (or to “sentences” in short). Clearly, at least from the viewpoint of natural language, the definition of the (finite) alphabet is immediate. Even the lexicon can be defined, at least to a first approximation, in a simple fashion. We will be content, for the time being, with a finite vocabulary; we can simply list the words of interest to us. This is certainly what happens with natural languages, given that dictionaries are finite volumes!²

How do we describe the syntax? In a natural language, it is the same natural language which is used, in general, to describe its own syntax (classical examples are grammar textbooks for various languages). Also a syntax for a programming language is often described using natural language, particularly for the older languages. Natural language, though, often introduces ambiguity in its syntactic description and, in particular, it is of no use in the process of translating (compiling) a program written in a high-level language into another (generally, at a lower level) language.

Linguistics, on the other hand, through the work in the 1950s of the American linguist Noam Chomsky, has developed techniques to describe syntactic phenomena in a formal manner. This description uses formalisms designed specifically to limit the ambiguity that is always present in natural language. These techniques, known as generative grammar, are not of much use in describing the syntax of natural languages (which are too complex and highly sophisticated). Instead, they are a fundamental tool for describing the syntax of programming languages (and are particularly useful for their compilation, as we will briefly see in Sect. 2.4).

Example 2.1 We will describe a simple language. It is the language of palindromic strings, composed of the symbols a and b .³ Let us therefore begin by fixing the *alphabet*, $A = \{a, b\}$. We must now select, from all strings over A (that is finite sequences of elements of A), the palindromic strings. The simplest way to do this is to observe that there is a simple recursive definition of a palindromic string. Indeed, we can say (this is the basis of the induction) that a and b are palindromic strings. If, then, s is any string that we know already to be palindromic, then so are asa and bsb (this is the induction step).

It is not difficult to convince ourselves that this definition captures all and only the palindromic strings of odd length over A . It remains to account for even-length strings, such as $abba$. To include these as well, we add the fact that the empty string (that is the string which contains no symbol from the alphabet) is also a palindromic string to the base case of the inductive definition. Now our definition categorises all

² In programming languages, the lexicon can also be formed from an infinite set of words. We will see below how this is possible.

³ A string is palindromic if it is identical to its mirror image. That is, the string is the same when read from left to right or right to left. A famous example in Latin is, ignoring spaces, the riddle “in girum imus nocte et consumimur igni”.

and only the palindromic strings over the alphabet A . If a string really is a palindromic string, for example $aabaa$, or $abba$, there exists a sequence of applications of the inductive rules just mentioned which will construct it. On the other hand, if a string is not a palindromic string (for example $aabab$), there is no way to construct it inductively using the rules just introduced. $\square$

Context-free grammars, which we introduce in the next section, are a notation for the concise and precise expression of recursive definitions of strings such as the one we have just seen. The inductive definition of palindromic strings can be expressed in grammatical form as:

$$\begin{aligned} P &\rightarrow \\ P &\rightarrow a \\ P &\rightarrow b \\ P &\rightarrow aPa \\ P &\rightarrow bPb \end{aligned}$$

In these rules, P stands for “any palindromic string”, while the arrow $\rightarrow$ is read as “can be”. The first three lines will be immediately recognised as the basis of the inductive definition, while the last two form the induction step.

2.2.1 Context-Free Grammars

The example just given is a *context-free grammar*, a fundamental tool for the description of programming languages. We begin by introducing a little notation and terminology. Having fixed a finite (or denumerable) set A , which we call the *alphabet*, we denote by A^* the set of all *finite strings over A* (that is finite-length sequences of elements of A ; the $*$ operator is called *Kleene’s star*). Let us immediately observe that, according to the definition, the sequence of length zero also belongs to A^* —this is the empty string, denoted by ϵ .

A *formal language over the alphabet A* is a subset of A^* . A formal grammar serves to identify a certain subset of strings from all those possible over a given alphabet.⁴

Definition 2.1 (*Context-Free Grammar*) A context-free grammar is a quadruple (NT, T, R, S) where:

1. NT is a finite set of symbols (non-terminal symbols, or variables, or syntactic categories).
2. T is a finite set of symbols (terminal symbols).

⁴ In formal languages, a sequence of terminal symbols which appears in a language is usually called a “word” of the language. We do not use this terminology; instead we speak of strings, to avoid ambiguity with words and phrases of a language (natural or artificial), in the sense in which they are normally understood.

3. R is a finite set of *productions* (or *rules*), each of which is composed of an expression of the form:

$$V \rightarrow w$$

where V (the *head* of the production) is a single non-terminal symbol and w (the *body*) is a string composed of zero or more terminal or non-terminal symbols (that is w is a string over $T \cup NT$).

4. S is an element of NT (the *initial symbol*).

According to this definition, therefore, the grammar in the example shown in Example 2.1 is composed of the quadruple $(\{P\}, \{a, b\}, R, P)$, where R is the set of productions used in the example.

Let us observe that, according to the definition, a production can have an empty body, that is, it can be composed of the null symbol, or, more properly, can be formed from the *empty string*, ϵ . We will see in a later section how a given grammar defines a language over the alphabet formed from its terminal symbols. Let us begin however with an example.

Example 2.2 Let us see a simple grammar that describes arithmetic expressions (over the operators $+$, $*$, $-$, both unary and binary). The atomic elements of these expressions are simple identifiers formed from finite sequences of the symbols a and b .

We define the grammar $G = (\{E, I\}, \{a, b, +, *, -, (,)\}, R, E)$, where R is the following set of productions:

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow E - E$
5. $E \rightarrow -E$
6. $E \rightarrow (E)$
7. $I \rightarrow a$
8. $I \rightarrow b$
9. $I \rightarrow Ia$
10. $I \rightarrow Ib$

Unlike the grammar in Example 2.1, this one has more than one non-terminal symbol corresponding, in an informal way, to an expression, E , or to an identifier, I . Note, once more, how the productions are a synthetic way to express recursive definitions. In this case, we are dealing with two definitions (which have been graphically separated), of which the one for E uses in its base case the non-terminal symbol inductively defined using the other recursive definition. $\square$

BNF

In the context of programming languages, context-free grammars were used for the first time in the definition of the language ALGOL 60. In the report that introduced

ALGOL 60, the grammar that defines the language is described using a notation that is slightly different from the one we have just introduced. This notation, (considered, among other things, to be usable with a reduced character set, that does not include arrow, italics, upper case, etc.) goes under the name of *Backus Naur normal form* (BNF), named after two authoritative members of the ALGOL committee (John Backus who had previously directed the Fortran project—the first high-level language—and Peter Naur). In BNF:

- The arrow “ $\rightarrow$ ” is replaced by “ $::=$ ”.
- The non-terminal symbols are written between angle brackets (for example $\langle Exp \rangle$ and $\langle Ide \rangle$ could be the two non-terminal symbols of the grammar of Example 2.2).
- Productions with the same head are grouped into a single block using vertical bars (“|”) to separate the productions. In Example 2.2, the productions for E could therefore be expressed as follows:

$$\langle E \rangle ::= \langle I \rangle \mid \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid \langle E \rangle - \langle E \rangle \mid -\langle E \rangle \mid (\langle E \rangle).$$

Derivations and Languages

A grammar inductively defines a set of strings. We can make explicit the operative aspect of this inductive definition using the concept of derivation.

Example 2.3 We can ensure that the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$ is correct according to the grammar of Example 2.2, reading the productions as “rewriting rules”, and by repeatedly applying them. We use a bigger arrow (“ $\Rightarrow$ ”) to denote the operation of string rewriting. We can then proceed as follows. We start with the initial symbol, E , and rewrite it using a production (which we are allowed to select). Let us, for example, use production (3), obtaining the rewriting $E \Rightarrow E * E$. Now we concentrate on the right-hand side of this production. We have two E ’s that can be rewritten (expanded) independently of each other. Let us take the one on the left and apply production (1) to obtain $E * E \Rightarrow I * E$. We can now choose whether we expand I or E (or the other way round). Figure 2.1 shows the rewriting that we have just started, developed until the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$ has been derived. The production used is represented by each $\Rightarrow$ ’s subscript. $\square$

We can capitalise on this example with the following definition.

Definition 2.2 (*Derivation*) Having fixed a grammar, $G = (NT, T, R, S)$, and assigned two strings, v and w over $NT \cup T$, we say that w is immediately derived from v (or: v is rewritten in a single step into w), if w is obtained from v by substituting the body of a production of R whose head is V for a non-terminal symbol, V , in v . In this case, we will write $v \Rightarrow w$.

We say that w is derived from v (or: v is rewritten to w) and we write $v \Rightarrow^* w$, if there exists a finite (possibly empty) sequence of immediate derivations $v \Rightarrow w_0 \Rightarrow w_1 \Rightarrow \dots \Rightarrow w$.

Fig. 2.1 A derivation of the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$

$$\begin{aligned}
 E &\Rightarrow_3 E * E \\
 &\Rightarrow_1 I * E \\
 &\Rightarrow_{10} I\mathbf{b} * E \\
 &\Rightarrow_7 \mathbf{ab} * E \\
 &\Rightarrow_6 \mathbf{ab} * (E) \\
 &\Rightarrow_2 \mathbf{ab} * (E + E) \\
 &\Rightarrow_1 \mathbf{ab} * (I + E) \\
 &\Rightarrow_7 \mathbf{ab} * (\mathbf{a} + E) \\
 &\Rightarrow_1 \mathbf{ab} * (\mathbf{a} + I) \\
 &\Rightarrow_8 \mathbf{ab} * (\mathbf{a} + \mathbf{b})
 \end{aligned}$$

Using the notation just introduced, and using the grammar for expressions that we have used so far we can write, for example $E * E \Rightarrow^* \mathbf{ab} * (\mathbf{a} + I)$. Particularly interesting are those derivations where on the left of the $\Rightarrow^*$ symbol there is the grammar's initial symbol and on the right is a string solely composed of *terminal* symbols. In a certain sense, these are maximal derivations which cannot be extended (by rewriting a non-terminal) either on the left or right. Following the intuition that has led us to the introduction of grammars, these derivations are the ones that give us the correct strings as far as the grammar is concerned.

Definition 2.3 (*Generated Language*) The language generated by a grammar $G = (NT, T, R, S)$ is the set $\mathcal{L}(G) = \{w \in T^* \mid S \Rightarrow^* w\}$.

Note that this definition, in accordance with everything we said at the start of Sect. 2.2.1, defines precisely a language over T^* .

Derivation Trees

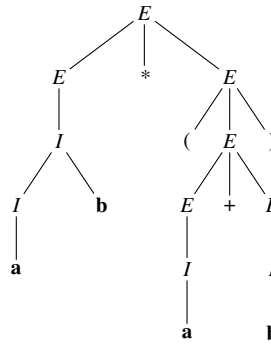
The derivation of a string is a sequential process. In it, there are steps that must be performed in a certain order. For example, in Fig. 2.1, it is clear that the first $\Rightarrow_{10}$ must follow the first $\Rightarrow_3$ because production (10) rewrites the non-terminal symbol I which does not exist in the initial string (which is composed of only the initial symbol E) which is introduced by production (3). But there are some steps whose order could be exchanged. In the derivation in Fig. 2.1, for example, each time that it is necessary to rewrite a non-terminal symbol, the leftmost is always chosen. We may, on the other hand, concentrate first on the rightmost non-terminal symbol, thereby obtaining the derivation shown in Fig. 2.2.

The two derivations that we have given for the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$ are, in an intuitive way, equivalent. Both reconstruct the structure of the string in the same way (in terms of non-terminal and terminal strings), while they differ only in the order in which the productions are applied. This fact is made explicit in Fig. 2.3 that represents the derivation of the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$ in the form of a tree.

Fig. 2.2 Another derivation
for the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$

$$\begin{aligned}
 E &\Rightarrow_3 E * E \\
 &\Rightarrow_6 E * (E) \\
 &\Rightarrow_2 E * (E + E) \\
 &\Rightarrow_1 E * (E + I) \\
 &\Rightarrow_8 E * (E + \mathbf{b}) \\
 &\Rightarrow_1 E * (I + \mathbf{b}) \\
 &\Rightarrow_7 E * (\mathbf{a} + \mathbf{b}) \\
 &\Rightarrow_1 I * (\mathbf{a} + \mathbf{b}) \\
 &\Rightarrow_{10} I\mathbf{b} * (\mathbf{a} + \mathbf{b}) \\
 &\Rightarrow_7 \mathbf{ab} * (\mathbf{a} + \mathbf{b})
 \end{aligned}$$

Fig. 2.3 A derivation tree
for the string $\mathbf{ab} * (\mathbf{a} + \mathbf{b})$



Definition 2.4 Given a grammar, $G = (NT, T, R, S)$, a derivation tree (or parse tree) is an ordered tree in which:

1. Each node is labelled with a symbol in $NT \cup T \cup \{\epsilon\}$.
2. The root is labelled with S .
3. Each interior node is labelled with a symbol in NT .
4. If a certain node has the label $A \in NT$ and its children are $m_1, \dots, m_k$ labelled respectively with $X_1, \dots, X_k$, where $X_i \in NT \cup T$ for all $i \in [1, k]$, then $A \rightarrow X_1 \dots X_k$ is a production of R .
5. If a node has label ϵ , then that node is the unique child of its parent A , and $A \rightarrow \epsilon$ is a production in R .

It is easy to see that, a derivation tree can be obtained from any derivation. It is enough to start at the root (labelled with the initial symbol) and successively adding a level of children corresponding to the production used during the derivation. By applying this procedure to the derivation of Fig. 2.1, the derivation tree of Fig. 2.3 is obtained. Let us now apply this procedure to the derivation in Fig. 2.2. Again, the tree in Fig. 2.3 is obtained. The two derivations result in the same tree, and this fact corresponds to the intuition that the two derivations were substantially equivalent.

Derivation trees are one of the most important techniques in the analysis of programming language syntax. The structure of the derivation tree indeed expresses, by

Trees

The concept of tree is of considerable importance in computer science and is also used a great deal in common language (think of a genealogical tree, for example). A tree is an information structure that can be defined in different ways and that exists in different “spaces”. For our purposes, we are interested only in ordered, rooted trees (or simply *trees*) which we can define as follows. A (rooted, ordered) tree is a finite set of elements called *nodes*, such that if it is not empty, a particular node is called *the root* and the remaining nodes, if they exist, are partitioned between the elements of an (ordered) n -tuple $\langle S_1, S_2, \dots, S_n \rangle$, $n \geq 0$, where each S_i , $i \in [1, N]$ is a tree.

Intuitively, therefore, a tree allows us to group nodes into levels where, at level 0, we have the root, at level 1 we have the roots of the trees $S_1, S_2, \dots, S_n$ and so on.

Another (equivalent) definition of tree is often used. It is probably more significant for the reader who is familiar with genealogical trees, botanical classifications, taxonomies, etc. According to this definition, a tree is a particular case of a graph: a (rooted, ordered) tree is therefore a pair $T = (N, A)$, where N is a finite set of *nodes* and A is a set of ordered pairs of nodes, called *arcs*, such that:

- The number of arcs is equal to one less than the number of nodes.
- T is *connected*, that is, for each pair of nodes, $n, m \in N$, there exists a sequence of distinct nodes $n_0, n_1, \dots, n_k$ such that $n_0 = n$ and $n_k = m$ and the pair (n_i, n_{i+1}) is an arc, for $i = 0, \dots, k-1$.
- A (unique) node, r , is said to be the *root* and the nodes are ordered by level according to the following inductive definition. The root is at level 0; if a node n is at level i , and there exists the arc $(n, m) \in A$ then node m is at level $i + 1$.
- Given a node, n , the nodes m such that there exists an arc $(n, m) \in A$ are said to be the *children* of n (and n is said to be their *parent*); for every node $n \in N$, a total order is established on the set of all the children of n .

Using a curious mix of botanical, mathematical and “familiar” terms, nodes with the same parent are said to be *siblings* while nodes without children are said to be *leaves*. The root is the only node without a parent.

means of its subtrees, the logical structure that the grammar assigns to the string. For example, the tree in Fig. 2.4 corresponds to the following derivation:

$$\begin{aligned}
 E &\Rightarrow_2 E + E \\
 &\Rightarrow_3 E * E + E \\
 &\Rightarrow_1 I * E + E \\
 &\Rightarrow_7 \mathbf{a} * E + E \\
 &\Rightarrow_1 \mathbf{a} * I + E \\
 &\Rightarrow_8 \mathbf{a} * \mathbf{b} + E \\
 &\Rightarrow_1 \mathbf{a} * \mathbf{b} + I \\
 &\Rightarrow_7 \mathbf{a} * \mathbf{b} + \mathbf{a}
 \end{aligned}$$

In the tree in Fig. 2.4, we can see that the subexpression $\mathbf{a} * \mathbf{b}$ appears as a left child of a sum. This expresses the fact that the string $\mathbf{a} * \mathbf{b} + \mathbf{a}$ must be interpreted as “first multiply $\mathbf{a}$ by $\mathbf{b}$ and add the result to $\mathbf{a}$ ”, given that in order to compute the sum, the operand present in the left subtree is needed.

Fig. 2.4 A derivation tree for the string $\mathbf{a * b + a}$

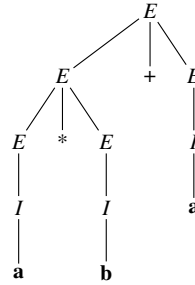
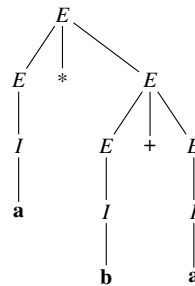


Fig. 2.5 Another derivation tree for the string $\mathbf{a * b + a}$



Ambiguity

Let us consider the following derivation (again using the grammar of the example on Sect. 2.2.1):

$$\begin{aligned}
 E &\Rightarrow_3 E * E \\
 &\Rightarrow_1 I * E \\
 &\Rightarrow_7 \mathbf{a} * E \\
 &\Rightarrow_2 \mathbf{a} * E + E \\
 &\Rightarrow_1 \mathbf{a} * I + E \\
 &\Rightarrow_8 \mathbf{a} * \mathbf{b} + E \\
 &\Rightarrow_1 \mathbf{a} * \mathbf{b} + I \\
 &\Rightarrow_7 \mathbf{a} * \mathbf{b} + \mathbf{a}
 \end{aligned}$$

If we construct the corresponding derivation tree, we obtain the tree in Fig. 2.5. Comparing the trees in Figs. 2.4 and 2.5, we see that we have two different trees producing the same string. Reading the leaves of the two trees from left to right, in both cases, we obtain the string $\mathbf{a * b + a}$. Let us observe, however, that the two trees are radically different. The second assigns a structure to the same string that is quite different (and therefore reveals a different precedence implicit in the arithmetic operators). If we want to use derivation trees to describe the logical structure of a string, we are in a bad position. The grammar of Example 2.2 is incapable of assigning a unique structure to the string in question. According to how the derivation is constructed, the precedence of the two arithmetic operators differs. Let us now see a formal definition of these concepts.

First, let us be clear about what it means to read or visit the leaves of a tree “from left to right”. Visiting a tree consists of following a path through the tree’s nodes so that each node is examined exactly once. There are many ways to visit nodes (the most common being depth-first, breadth-first, and symmetric visits), which, when applied to the same tree, produce different sequences of nodes. If only the leaves are considered, however, each and every one of these methods produces the same result for which we can use the following definitions:

Definition 2.5 Let $T = (N, A)$, be a non-empty, ordered tree with root r . The result of the left-to-right traversal of T is the sequence of nodes (actually: leaves) obtained by the following recursive definition:

- If r has no children, the result of the traversal is r .
- If r has k children, $m_1, \dots, m_k$, let $T_1, \dots, T_k$ be the subtrees of T such that T_i has m_i as its root (T_i therefore contains m_i and all of that part of T which is underneath this node). The result of the traversal is the sequence of nodes obtained by visiting T_1 to T_k in turn, from left to right.

At this point, we can say what it means for a string to admit a derivation tree.

Definition 2.6 We say that a string of characters admits a derivation tree T if it is the result of a left-to-right traversal of T .

Finally, we can give the definition that interests us.

Definition 2.7 (*Ambiguity*) A grammar, G , is *ambiguous* if there exists at least one string of $\mathcal{L}(G)$ which admits more than one derivation tree.

We remark that ambiguity comes not from the existence of many derivations for the same string (a common and innocuous property) but from the fact that (at least) one string has more than one derivation *tree*.

An ambiguous grammar is useless as description of a programming language because it cannot be used to translate (compile) a program in a unique fashion. Fortunately, given an ambiguous grammar it is often possible to transform it into another, unambiguous, grammar that generates the same language.⁵ Techniques for grammar disambiguation are outside the scope of this book. By way of example, Fig. 2.6 shows an unambiguous grammar whose generated language coincides with the one given in the example on Sect. 2.2.1.

This grammar interprets the structure of an expression according to the usual concept of arithmetic operator precedence. Unary minus (“−”) has the highest precedence, followed by $*$, followed in their turn by $+$ and binary $-$ (which have the same

⁵ There exist pathological cases of languages which are generated only by ambiguous grammars. These languages have no relevance to programming languages.

$$G' = (\{E, T, A, I\}, \{\mathbf{a}, \mathbf{b}, +, *, -, (,)\}, R', E)$$

$$\begin{aligned} E &\rightarrow T \mid T + E \mid T - E \\ T &\rightarrow A \mid A * T \\ A &\rightarrow I \mid -A \mid (E) \\ I &\rightarrow \mathbf{a} \mid \mathbf{b} \mid I\mathbf{a} \mid I\mathbf{b} \end{aligned}$$

Fig. 2.6 An unambiguous grammar for the language of expressions

```

Statement ::= ... | IfThenStatement | IfThenElseStatement |
               StatementWithoutTrailingSubstatement
StatementWithoutTrailingSubstatement ::= ... | Block | EmptyStatement |
               ReturnStatement
StatementNoShortIf ::= ... | StatementWithoutTrailingSubstatement |
               IfThenElseStatementNoShortIf
IfThenStatement ::= if ( Expression ) Statement
IfThenElseStatement ::=
    if ( Expression ) StatementNoShortIf else Statement
IfThenElseStatementNoShortIf ::=
    if ( Expression ) StatementNoShortIf else StatementNoShortIf

```

Fig. 2.7 Grammar for Java conditional commands

precedence). The grammar interprets, then, a sequence of operators at the same level of precedence by association to the right. For example, the string $\mathbf{a} + \mathbf{b} + \mathbf{a}$ will be assigned the same structure as the string $\mathbf{a} + (\mathbf{b} + \mathbf{a})$. The absence of ambiguity is paid for by increased complexity. There are more non-terminals and the intuitive interpretation of the grammar is therefore more difficult.

The need to provide unambiguous grammars explains the contortions that appear in the definitions of some programming languages. For example, Fig. 2.7 shows an extract of the official grammar for Java's conditional command (its *if*) (the non-terminal symbols are printed in italic, while the terminal symbols in this extract are *if*, *else* and brackets).

This grammar is interesting for two reasons. First, note that what are formally considered single symbols (terminal or non-terminal) in context-free grammars are here represented by words composed of a number of characters. For example, *if* represents a single terminal symbol and, analogously, *IfThenElseStatement* represents a non-terminal symbol.

This happens because, in the definition of a programming language, it is preferential to use meaningful words (*if*, *then*, *else*) which can, up to certain limits, suggest an intuitive meaning, rather than symbols which would be harder to understand by the language user. In other words, as we will better see in the next chapters, that the use of (meaningful) symbolic names definitely makes programming easier. Analogously, for non-terminal symbols, they make the grammar easier to understand. It is definitely better to use names such as *Statement* rather than single symbols.

The second interesting aspect of the grammar is its complex nature. This complexity is necessary to resolve the ambiguity of strings (programs) such as the one exemplified by the following skeleton program:

```
if (expression1) if (expression2) command1;  
    else command2;
```

Java, like a great many other programming languages allows conditional commands both with an `else` branch and without it. When an `if` command without `else` is combined with an `if` with an `else` branch (as in the case of the program appearing above), it is necessary to determine which of the two `ifs` is the owner of the single `else`. The grammar in Fig. 2.7 is an unambiguous one which makes “an `else` clause belong to the innermost `if` to which it might possibly belong” (from the Java definition [2]). In simple words, the `else` is paired with the second `if` (the one which tests `expression2`). Intuitively, this happens because, once an `if` command with an `else` is introduced by use of the non-terminal symbol *IfThenElseStatement*, the command in the *then* branch will be unable to hold an `if` command without an `else`, as can be seen by inspection of the rules which define the non-terminal symbol *StatementNoShortIf*.

2.3 Contextual Syntactic Constraints

The syntactic correctness of a phrase of a programming language sometimes depends on the *context* in which that phrase occurs. Let us consider, for example, the assignment `I = R+3`; and assume that indeed the productions in the grammar allow the correct derivation of this string. It might be the case, though, that the assignment is incorrect at the position in the program at which it occurs. For example, if the language requires the declaration of variables, it is necessary for programs to contain the declarations of `I` and `R` before the assignment. Moreover, if the language is statically typed, it could be necessary that the type of the expression `R+3` is compatible⁶ with that of the variable `I`.

Strings that are correct with respect to the grammar, therefore, could be legal only *in a given context*. There are many examples of these contextual syntactic constraints:

- An identifier must be declared before use (Pascal, C, Java).
- The number of actual parameters to a function must be the same as the formal parameters (C, Pascal, Java, etc.).
- In the case of assignment, the type of an expression must be compatible with that of the variable to which it is assigned (C, Pascal, Java, etc.).
- Assignments to the control variable of a `for` loop are not permitted (Pascal).
- Before using a variable, there must have been an assignment to it (Java, Python).
- A method can be redefined (*overridden*) only by a method with the same signature (Java) or with a compatible signature (Java5).
- ...

⁶ All these concepts will be introduced below. For now, an intuitive understanding of these concepts will suffice.

These are certainly syntactic constraints. However, their contextual nature makes it impossible to describe them using a context-free grammar (the name of this class of grammars was chosen on purpose). A book on formal languages is required to go into detail on these issues; here, it suffices to note that there exist other types of grammar, called contextual grammars.⁷ These grammars permit the description of cases such as this. These grammars are difficult to write, and to process. Moreover, there are no automatic techniques for efficient generation of translators, such as exist, on the other hand, for context-free grammars. This suggests, therefore, that use of grammars should be limited to the non-contextual description of syntax and then to express the additional contextual constraints using natural language or using formal techniques such as transition systems (which we will see in Sect. 2.5 when we consider semantics).

In programming language terminology, these contextual constraints that appear in the syntactic aspects of a language are usually called *static semantic* constraint. In the jargon, “syntax” usually means “describable using a context-free grammar”, “static semantics” means “describable with verifiable contextual constraints on a static basis using the program text”, while “dynamic semantics” (or simply “semantics”) refers to everything that happens when a program is executed.

The distinction between context-free and context-dependent syntax (that is, static semantics) is established precisely by the expressive power of context-free grammars. The distinction between syntax and semantics is not always clear. Some cases are clear. Let us assume, for example, that we have a language whose definition establishes that should a division by 0 happen during execution, the abstract machine for the language *must* report it with an explicit error. This is clearly a dynamic semantic constraint. But let us assume, now, that the definition of another language is specified as follows:

A program is syntactically incorrect when a division by 0 *can* happen.

The program in Fig. 2.8 would be syntactically incorrect in this hypothetical language because there exists a sequence of executions (those in which the `read` command assigns the value 0 to `A`) which causes division by zero. Does the constraint that we have stated above (which certainly cannot be expressed in a free grammar) belong to static or dynamic semantics? It is clear that it refers to a dynamic event (division by zero) but the constraint, to be meaningful (that is, that it really does

Fig. 2.8 A program that can cause a division by 0

```
int A, B;
read(A);
B = 10/A;
```

⁷ In extreme synthesis, in a contextual grammar, a production can take the form (which is more general than in a context-free grammar) $uAv \rightarrow uvw$, where u , v and w are strings over $T \cup NT$. Fundamental to this production, the non-terminal symbol A can be rewritten to w only if it appears in a certain context (the one described by u and v).

exclude some programs) would have to be detected statically (as we have done for the simple program given above). A person can decide that it is syntactically incorrect without executing it but only from looking at the code and reasoning about it. More important than its classification, is understanding whether we have a verifiable constraint or not. Otherwise stated, is it really possible to implement an abstract machine for such a language, or does the very nature of the constraint imply that there can exist no abstract machine which is able to check it in a static fashion for an arbitrary program?

These are questions that we cannot immediately answer, even though they are of paramount importance, because they concern what we can, and cannot, do with an abstract machine. We will touch briefly these issues in the next chapter (Chap. 3).

2.4 Compilers

We may finally see how the syntactic description of a language can be used automatically to translate a program. We know from Chap. 1 that such an automatic translator is called a *compiler*, whose general logical structure is shown in Fig. 2.10. It is characterised by a cascaded series of phases. The various phases, starting with a string representing the program in the *source* language, generate various internal intermediate representations until a string in the *object* language is generated. Note that, as seen in the previous chapter, in this context, “object language” does not necessarily equate to “machine code” or to “low-level language”. It is merely a language towards which the translation is directed. In what follows, we briefly describe the different phases of compilation. We do not pretend to explain how to construct a compiler (see [3]), but just to fix some ideas useful for our study of programming languages.

Lexical Analysis

The aim of lexical analysis is to read the symbols (characters) forming the program sequentially from the input and to group these symbols into meaningful logical units, which we call *tokens* (which are analogous, for our programming language, to the words in the dictionary of a natural language). For example, the lexical analyser of C or Java, when presented with the string `x = 1 + foo++;` will produce 7 tokens: the identifier `x`, the assignment operator `=`, the number `1`, the addition operator `+`, the identifier `foo`, the auto increment operator `++` and finally the command termination token `;`. Particularly important tokens are the reserved words of the language (such as `for`, `if`, `else`, etc.), operators, open and close brackets (such as `{` and `}` in C or Java, but also `indent` and `unindent` in Python). Lexical analysis (or *scanning*) is a very simple operation which scans the input text of the source program from left to right, taking a single pass to recognise tokens. No check is yet made on the sequence of tokens such as, for example, checking that brackets are correctly balanced. As discussed in detail in the box, the technical tool used for lexical analysis is a particular

Abstract and Concrete Syntax

The grammar of a programming language defines the language as a set of strings. This set of strings corresponds in a natural way to the set of their derivation trees. These trees are much more interesting than the strings. First, they abstract from the specific lexical characteristic of the tokens. It can also happen that lexically different structures can result in the same tree. The tree in Fig. 2.9 could correspond to the Python string:

```
if A==0: X=0
else: X=1
```

or to the Java string:

```
if (A==0) X=0; else X=1;
```

As we have already repeatedly observed, derivation trees are interesting because they express the canonical structure that can be assigned to the string.

Not all derivation trees correspond to legal programs. We know that static semantic analysis has the task of selecting those trees satisfying the contextual constraints of the language. The set of trees resulting from this process constitutes the *abstract syntax* of language. This is a much more convenient way of thinking of a language if we are interested in its manipulation (and not merely in writing correct programs in it).

class of generative grammars (regular grammars). The use of a grammar to describe the elements of the lexicon is due both to the necessity of efficiently recognising these elements, and to the fact that, unlike the case of natural language lexicons, lexicons for programming languages can be formed from an infinite set of words and therefore a simple list, so the one used in a normal dictionary is inadequate (think, for example, of the possibility of defining identifiers as sequences of characters of arbitrary length starting with a particular letter).

Syntactic Analysis

Once the list of tokens has been constructed, the syntactic analyser (or *parser*) seeks to construct a derivation tree for this list. This will be a derivation tree in the grammar of the language. Each leaf of this tree must correspond to a token from the list obtained by the scanner. Moreover, these leaves, read from left to right, must form a correct phrase (or a sequence of terminal symbols) in the language. We already know that

Fig. 2.9 An abstract syntax tree

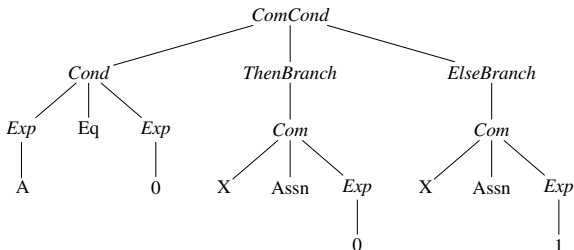
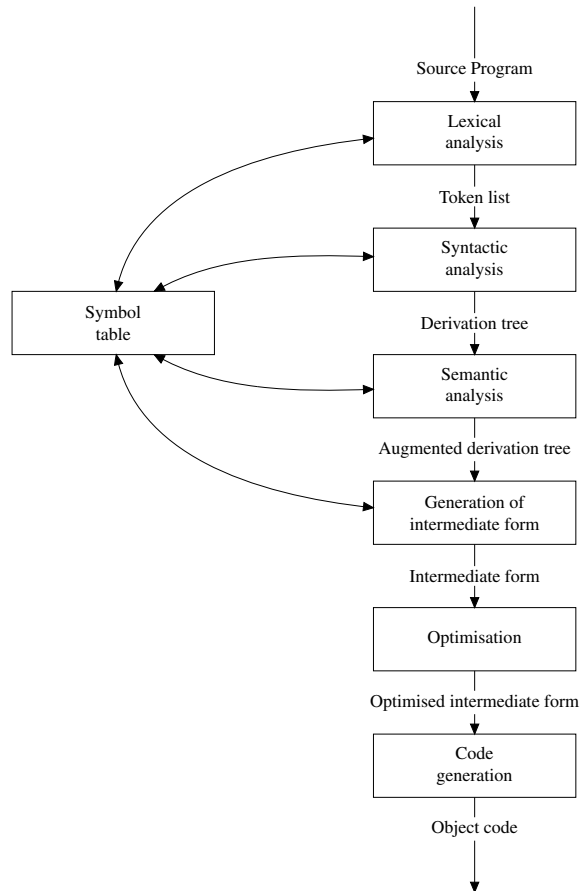


Fig. 2.10 Organisation of a compiler



such trees represent the logical structure of the program which will be employed by the successive phases of compilation.

It can happen that the parser is unable to construct a derivation tree. This happens when the input string is not correct with reference to the language's grammar. In such a case, the parser reports the errors it has encountered and compilation is aborted. In reality, lexical and syntactic analysis are interleaved in a more direct fashion than appears from these notes (in particular, the two phases are not sequential but the scanner produces a token whenever the parser requests for one).

Semantic Analysis

The derivation tree (which represents the syntactic correctness of the input string) is subjected to checks of the language's various context-based constraints. As we have seen, it is at this stage that declarations, types, number of function parameters, etc., are processed. As these checks are performed, the derivation tree is *augmented* with the information derived from them and new structures are generated. By way of

Regular Grammars

The difference between lexical and syntactic analysis can be made precise using a formal classification of grammars. If a grammar has all productions of the form $A \rightarrow bB$ (or in the form $A \rightarrow Bb$), where A and B are non-terminal symbols (B can also be absent or coincide with A) and b is a single terminal symbol, the grammar is said to be *regular*. In Example 2.2, the (sub-) grammar based on the non-terminal symbol I is a regular grammar (while the subgrammar for E is not).

The expressive power of regular grammars is highly limited. In particular, using a regular grammar, it is not possible to “count” an arbitrary number of characters. As a consequence, it is not possible to express the balancing of syntactic structures such as brackets using a regular grammar.

Technically, lexical analysis is the first phase of translation which checks that the input string can be decomposed into tokens, each of which is described by a regular grammar (in Example 2.2, lexical analysis would have recognised the sequences of **a** and **b** as instances of the non-terminal symbol I).

Once the sequence of tokens has been obtained, syntactic analysis takes place using a properly context-free grammar which uses the tokens produced by the preceding lexical analysis as its terminal symbols.

an example, every token that corresponds to a variable identifier will be associated with its type, the place of declaration and other useful information (for example its *scope*, see Chap. 4). To avoid useless duplication, this information is, in general, collected into structures external to the derivation tree. Among these structures, the one in which information about identifiers is collected is called the symbol table. The symbol table plays an essential role in successive phases.

At the cost of boring the reader, so that we do not confuse it with what we call semantics in Sect. 2.5, let us note that the term semantic analysis is an historical relic and that it is concerned with context-based syntactic constraints.

Generation of Intermediate Forms

An appropriate traversal of the augmented derivation tree allows an initial phase of code generation. It is not yet possible to generate code in the object language, given that there are many optimisations left to do and they are independent of the specific object language. Moreover, a compiler is often implemented to generate code for a whole series of object languages (for example, machine code for different architectures), not just one. It is useful, therefore, to concentrate all choices relating to a specific language in a single phase and to generate code to an intermediate form, which is designed to be independent of both the source and the object languages.

Code Optimisation

The code obtained from the preceding phases by repeatedly traversing the derivation tree is fairly inefficient. There are many optimisations that can be made before generating object code. Typical operations that can be performed are:

- *Dead code removal*. That is, removal of pieces of code that can never be executed because there is no execution sequence that can reach them.
- *In-line expansion* of function calls. Some function (procedure) calls can be substituted by the body of the associated function, making execution faster. It also makes other optimisations possible.
- Subexpression factorisation. Some programs compute the same value more than once. If, and when, this fact is discovered by the compiler, the value of the common subexpression can be calculated once only and then stored.
- Loop optimisations. Iterations are the places where the biggest optimisations can be performed. Among these, the most common consists of removing from inside a loop the computation of subexpressions whose value remains constant during different iterations.

Code Generation

Starting with optimised intermediate form, the final object code is generated. There follows, in general, a last phase of optimisation which depends upon the specific characteristics of the object language. In a compiler that generates machine code, an important part of this last phase is register assignment (decisions as to which variables should be stored in which processor registers). This is a choice of enormous importance for the efficiency of the final program.

2.5 Semantics

The description of the semantics of a programming language is more complex than its syntactic description. This complexity is due both to technical problems, as well to the need to mediate between two opposing issues: the need for exactness as well as for flexibility. As far as exactness is concerned, a precise and unambiguous description is required of what must be expected from every syntactically correct construct so that every user knows a priori (that is before program execution), and in a manner independent of the architecture, what will happen at runtime. This search for exactness, however, must not preclude different implementations of the same language, all of which are correct with respect to the semantics. The semantic specification must therefore also be flexible, that is, it must not anticipate choices that are to be made when the language is implemented (and which therefore do not play a part in the definition of the language itself).

It would not be difficult to achieve exactness at the expense of flexibility. It is enough to give the semantics of a language using a specific compiler on a specific physical architecture. The official meaning of a program is given by its execution on this architecture after translation using that compiler. This solution has clearly no flexibility—there exists only one implementation (the official one) and all the other implementations are completely equivalent. But to what level of detail is the canoni-

cal implementation normative? Is the computation time of a particular program part of its definition? Is the reporting of errors? How can typically architecture-dependent input/output commands be ported to a different architecture? When the implementation technology of the physical architecture changes, how does the semantics change?

One of the difficulties of a semantic definition is that of finding the happy medium between exactness and flexibility, in such a way as to remove ambiguity, still leaving room for implementation. This situation suggests using formal methods to describe the semantics. Methods of this kind have existed for a long time for the artificial languages of mathematical logic and they have been adapted by computer scientists for their own ends. Yet, some semantic phenomena make formal descriptions complex and not easily usable by anyone who does not have the appropriate skills. It is for this reason that the majority of official programming language definitions use natural language for their semantic description. This does not detract from the fact that formal methods for semantics are often used in the preparatory phases of the design of a programming language, or to describe some of its particular characteristics, where it is paramount to avoid ambiguity at the cost of simplicity.

Formal methods for semantics divide into two main families: *denotational* and *operational* semantics.⁸ Denotational semantics is the application to programming languages of techniques developed for the semantics of logic-mathematical languages. The meaning of a program is given by a function that expresses the input/output behaviour of the program itself. The domain and codomain of this function are suitable mathematical structures, such as the environment and the memory (store), that are internal to the language. Refined mathematical techniques (continuity, fixed points, etc.) allow the appropriate treatment of phenomena such as iteration and recursion (see [4]).

In the operational approach, on the other hand, there are no external entities (for example, functions) associated with language constructs. Using appropriate methods, an operational semantics specifies the behaviour of the abstract machine. That is, it formally defines the interpreter, making reference to an abstract formalism at a much lower level. The various operational techniques differ (sometimes profoundly) in their choice of formalism. Some semantics use formal automata, others use systems of logic-mathematical rules, yet others prefer transition systems to specify the state transformations induced by a program.

This is not a book in which a treatment of these semantic techniques can be given. We content ourselves with a simple example of techniques based on transition systems to give semantics for a rudimentary imperative language. This technique, called SOS (Structured Operational Semantics, [5]), is highly versatile and has been effectively used in giving the semantics to some languages including Standard ML (an influential functional language).

Figure 2.11 shows the grammar of a simple language. For reasons of readability (and so that we can obtain a concise semantics), we have a grammar with infinite

⁸ For completeness, we should also talk about algebraic and axiomatic semantics, but simplicity and conciseness require us to ignore them.

Fig. 2.11 Syntax of a simple imperative language
$$\begin{aligned}
Num &::= 1 \mid 2 \mid 3 \mid \dots \\
Var &::= X_1 \mid X_2 \mid X_3 \mid \dots \\
AExp &::= Num \mid Var \mid (AExp + AExp) \mid (AExp - AExp) \\
BExp &::= \mathbf{tt} \mid \mathbf{ff} \mid (AExp == AExp) \mid \neg BExp \mid (BExp \wedge BExp) \\
Com &::= \mathbf{skip} \mid Var := AExp \mid Com; Com \mid \\
&\quad \mathbf{if} BExp \mathbf{then} Com \mathbf{else} Com \mid \mathbf{while} BExp \mathbf{do} Com
\end{aligned}$$

productions for the *Num* and *Var* non-terminal symbols. The explicit presence of brackets around each binary operator application eliminates possible problems of ambiguity.

We need to introduce some notation. We will use n to denote an arbitrary *Num* (numeric constant); using X , we will denote an arbitrary *Var* (variable). We write a for an arbitrary *AExp* (arithmetic expression). We use b to denote an arbitrary *BExp* (boolean expression, where **tt** and **ff** denote the values *true* and *false*, respectively). We write c for arbitrary *Comm* (commands). We use, when needed, subscripts to distinguish between objects of the same syntactic category (for example, we will write a_1, a_2 , etc., for *AExps*).

State

The semantics of a command in our language uses a simple memory model which stores the values associated with the *Vars*. In this model, a *state* is a finite sequence of pairs of the form (X, n) which we can read as “in the current state, the variable X has the value n ” (in our little language, the value is always an integer). Given a command c (that is a derivation tree that is correct according to the grammar in Fig. 2.11), its reference state is given by a sequence of pairs which includes all *Vars* which are named in c . We denote an arbitrary state by σ or τ , with or without subscripts.

We need to define some operations on states: modification of an existing state and retrieval of a variable’s value in the current state. To this end, given a state, σ , a *Var* X and a value v , we write $\sigma[X \leftarrow v]$ to denote a new state that is the same as σ but differs from it by associating X with the value v (and thereby losing any previous association to X , if any). Given a state, σ , and a variable, X , we write $\sigma(X)$ for the value that σ associates with X ; this value is undefined if X does not appear in the domain of σ (σ is therefore a partial function).

Example 2.4 Let us fix $\sigma = [(X, 3), (Y, 5)]$, we have $\sigma[X \leftarrow 7] = [(X, 7), (Y, 5)]$. We also have $\sigma(Y) = 5$ and $\sigma[X \leftarrow 7](X) = 7$; $\sigma(W)$ is undefined. $\square$

Transitions

Structured operational semantics can be seen as an elegant way of defining the behavior of an abstract machine without going into any details about its implementation.⁹ This behavior is expressed in terms of the elementary computational steps of the abstract machine. The formal way in which structured operational semantics defines the meaning of a program, c , is in terms of a *transition* which expresses a step in the transformation (of the state and/or of the program itself) induced by the execution of the command. The simplest form of transition is:

$$\langle c, \sigma \rangle \rightarrow \tau,$$

where c is a command, σ the *starting state* and τ the *terminal state*. The interpretation that we give to this notation is that if we start the execution of c in the state σ , the execution terminates (in a single step) with state τ . For example, the transition which defines the **skip** command is

$$\langle \text{skip}, \sigma \rangle \rightarrow \sigma.$$

Starting in any state, σ , **skip** terminates leaving the state unchanged.

In more complex cases, a *terminal situation* will be reached not in a single step, but rather in many steps which progressively transform the state. The command, c , is progressively executed a part at a time until the whole has been “consumed”. These little steps are expressed by transitions of the form:

$$\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle.$$

For example, one of the transitions which define the conditional command will be:

$$\langle \text{if } tt \text{ then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle,$$

This means that if the boolean condition is true, the command in the **then** branch must be executed. Some transitions, finally, are *conditional*: if some command, c_1 , has a transition, then the command c has another transition. Conditional transitions take the form of a *rule*, expressed as a fraction:

$$\frac{\langle c_1, \sigma_1 \rangle \rightarrow \langle c'_1, \sigma'_1 \rangle \quad \langle c_2, \sigma_2 \rangle \rightarrow \langle c'_2, \sigma'_2 \rangle}{\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle}.$$

We read this rule in the following way. If the command c_1 , starting in state σ_1 , can perform a computational step that transforms itself into command c'_1 in state σ'_1 , and if the command c_2 , starting in σ_2 , can perform a computational step and transform itself into the command c'_2 in state σ'_2 , then the command c , starting in the state σ can perform a computational step and transform itself into the command c' in state σ' . It is clear that a specific rule will express a number of meaningful relationships between c , c_1 and c_2 (and their respective states). In general, c_1 and c_2 will be subcommands of c .¹⁰

⁹ Using terminology that we will use again in the final chapters of this book, we can say that it is a *declarative* description of the language’s abstract machine.

¹⁰ What we have called conditional rules are the inductive definition of the transition relation $\rightarrow$.

$$\begin{array}{c}
\langle X, \sigma \rangle \rightarrow \langle \sigma(X), \sigma \rangle \\
\\
\begin{array}{cc}
\langle (n+m), \sigma \rangle \rightarrow \langle p, \sigma \rangle & \langle (n-m), \sigma \rangle \rightarrow \langle p, \sigma \rangle \\
\text{where } p = n+m & \text{where } p = n-m \text{ e } n \geq m
\end{array} \\
\\
\frac{\langle a_1, \sigma \rangle \rightarrow \langle a', \sigma \rangle}{\langle (a_1 + a_2), \sigma \rangle \rightarrow \langle (a' + a_2), \sigma \rangle} & \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'', \sigma \rangle}{\langle (a_1 + a_2), \sigma \rangle \rightarrow \langle (a_1 + a''), \sigma \rangle} \\
\\
\frac{\langle a_1, \sigma \rangle \rightarrow \langle a', \sigma \rangle}{\langle (a_1 - a_2), \sigma \rangle \rightarrow \langle (a' - a_2), \sigma \rangle} & \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'', \sigma \rangle}{\langle (a_1 - a_2), \sigma \rangle \rightarrow \langle (a_1 - a''), \sigma \rangle}
\end{array}$$

Fig. 2.12 Semantics of arithmetic expressions

Expression Semantics

Figure 2.12 shows the rules for the semantics of arithmetic expressions. The first three rules are terminal rules (that is, the computation to the right of the arrow is in a form to which no more transitions can be applied). In order of presentation, they define the semantics of a variable, of addition in the case in which both of the summands are constants, of subtraction in the case in which both its arguments are constants and its result is a natural number. Note that no semantics is given for expressions such as $5-7$. The second group of four rules is formed of conditional rules. The first pair defines the semantics of sums and expresses the fact that to calculate the value of a sum, it is necessary first to evaluate its two arguments separately. The rules do *not* specify the *order of evaluation* for the arguments of an addition or a subtraction.

In our simple language, the evaluation of an expression does not modify the state. The state is present in the transition rules only because it is used to obtain the value of a name, in the first rule. In more realistic languages, expressions may also modify the state, by side effects.

Figure 2.13 shows the semantics of logical expressions, along the same ideas used for arithmetic expressions. In this figure, bv denotes a constant boolean value (**tt** or **ff**). The premises of the two rules of the fourth line (those for the evaluation of the equality comparison, $==$) are transitions for arithmetic expressions.

Command Semantics

We have noted that the state, σ , always remains unaltered during the evaluation of an expression. This changes for the semantics of commands as shown in Fig. 2.14. Rules (c1), (c2), (c6), and (c7) are terminal ones: the transition is of the form $\langle c, \sigma \rangle \rightarrow \tau$, with only a state on the right of $\rightarrow$. The premise of rule (c4) is always a terminal transition. The other rules (which define a transition of the form $\langle c, \sigma \rangle \rightarrow \langle c', \tau \rangle$) make the computation of a complex command proceed, by evaluating one of its subcommands (or subexpressions).

$$\begin{array}{c}
\frac{}{\langle (n == m), \sigma \rangle \rightarrow \langle \mathbf{tt}, \sigma \rangle \quad \text{if } n = m} \qquad \frac{}{\langle (n == m), \sigma \rangle \rightarrow \langle \mathbf{ff}, \sigma \rangle \quad \text{if } n \neq m} \\
\\
\frac{}{\langle (bv_1 \wedge bv_2), \sigma \rangle \rightarrow \langle bv, \sigma \rangle} \quad \text{where } bv \text{ is the and of } bv_1 \text{ and } bv_2 \\
\\
\frac{}{\langle \neg \mathbf{tt}, \sigma \rangle \rightarrow \langle \mathbf{ff}, \sigma \rangle} \qquad \frac{}{\langle \neg \mathbf{ff}, \sigma \rangle \rightarrow \langle \mathbf{tt}, \sigma \rangle} \\
\\
\frac{\langle a_1, \sigma \rangle \rightarrow \langle a', \sigma \rangle}{\langle (a_1 == a_2), \sigma \rangle \rightarrow \langle (a' == a_2), \sigma \rangle} \quad \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'', \sigma \rangle}{\langle (a_1 == a_2), \sigma \rangle \rightarrow \langle (a_1 == a''), \sigma \rangle} \\
\\
\frac{\langle b_1, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle (b_1 \wedge b_2), \sigma \rangle \rightarrow \langle (b' \wedge b_2), \sigma \rangle} \quad \frac{\langle b_2, \sigma \rangle \rightarrow \langle b'', \sigma \rangle}{\langle (b_1 \wedge b_2), \sigma \rangle \rightarrow \langle (b_1 \wedge b''), \sigma \rangle} \\
\\
\frac{\langle b, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle \neg b, \sigma \rangle \rightarrow \langle \neg b', \sigma \rangle}
\end{array}$$

Fig.2.13 Semantics of boolean expressions

$$\begin{array}{c}
\langle \mathbf{skip}, \sigma \rangle \rightarrow \sigma \quad (c1) \\
\\
\langle X := n, \sigma \rangle \rightarrow \sigma[X \leftarrow n] \quad (c2) \qquad \frac{\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle}{\langle X := a, \sigma \rangle \rightarrow \langle X := a', \sigma \rangle} \quad (c3) \\
\\
\frac{\langle c_1, \sigma \rangle \rightarrow \sigma'}{\langle c_1; c_2, \sigma \rangle \rightarrow \langle c_2, \sigma' \rangle} \quad (c4) \qquad \frac{\langle c_1, \sigma \rangle \rightarrow \langle c'_1, \sigma' \rangle}{\langle c_1; c_2, \sigma \rangle \rightarrow \langle c'_1; c_2, \sigma' \rangle} \quad (c5) \\
\\
\langle \mathbf{if tt then } c_1 \mathbf{ else } c_2, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle \quad (c6) \\
\langle \mathbf{if ff then } c_1 \mathbf{ else } c_2, \sigma \rangle \rightarrow \langle c_2, \sigma \rangle \quad (c7) \\
\\
\frac{\langle b, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2, \sigma \rangle \rightarrow \langle \mathbf{if } b' \mathbf{ then } c_1 \mathbf{ else } c_2, \sigma \rangle} \quad (c8) \\
\\
\langle \mathbf{while } b \mathbf{ do } c, \sigma \rangle \rightarrow \langle \mathbf{if } b \mathbf{ then } c; \mathbf{while } b \mathbf{ do } c \mathbf{ else skip}, \sigma \rangle \quad (c9)
\end{array}$$

Fig.2.14 Semantics of commands

Computations

A *computation* is a sequence of transitions which cannot be extended by another transition. Moreover, each transformation in a computation must be allowed by some rule.

Example 2.5 Let us consider the following program, c :

$$X := 1; \mathbf{while } \neg(X == 0) \mathbf{ do } X := (X - 1)$$

Let us fix a state which includes all the variables mentioned in the program, for example, $\sigma = [(X, 6)]$. We can calculate the computation of c in σ as follows. We

write c' to denote the iterative command **while** $\neg(X == 0)$ **do** $X := (X - 1)$. It is not difficult to see that the computation generated by c is the following:

$$\begin{aligned}
&\langle c, \sigma \rangle \\
&\rightarrow \langle c', \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if } \neg(X == 0) \text{ then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if } \neg(1 == 0) \text{ then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if tt then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle X := (X - 1); c', \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle X := (1 - 1); c', \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle X := 0; c', \sigma[X \leftarrow 1] \rangle \\
&\rightarrow \langle c', \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \langle \text{if } \neg(X == 0) \text{ then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \langle \text{if } \neg(0 == 0) \text{ then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \langle \text{if } \neg\text{tt then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \langle \text{if ff then } X := (X - 1); c' \text{ elseskip}, \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \langle \text{skip}, \sigma[X \leftarrow 0] \rangle \\
&\rightarrow \sigma[X \leftarrow 0]
\end{aligned}$$

The computation in the example is a *terminated* computation, in the sense that, after a finite number of transitions, we are left with a terminal transition, where no other transition is possible. However, the definition of computation does *not* require that a computation be finite but only that *it cannot be extended*. There could be, therefore, infinite computations, as in the following example:

Example 2.6 Consider the following program, d :

$X := 1; \text{ while } (X == 1) \text{ do skip}$

Assume we are in the state $\tau = [(X, 0)]$. Let d' be the command **while** $(X == 1)$ **do skip**. We have:

$$\begin{aligned}
&\langle d, \tau \rangle \\
&\rightarrow \langle d', \tau[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if } (X == 1) \text{ then skip ; } d' \text{ elseskip}, \tau[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if } (1 == 1) \text{ then skip ; } d' \text{ elseskip}, \tau[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{if tt then skip ; } d' \text{ elseskip}, \tau[X \leftarrow 1] \rangle \\
&\rightarrow \langle \text{skip ; } d', \tau[X \leftarrow 1] \rangle \\
&\rightarrow \langle d', \tau[X \leftarrow 1] \rangle \\
&\rightarrow \dots
\end{aligned}$$

□

There exist, therefore, two fundamentally different types of computations: *finite* ones (also called *terminating*) and *divergent* ones (that is, infinite ones which correspond, intuitively, to looping programs).

2.6 Pragmatics

The syntax and semantics of a programming language are always given with the most accurate precision. The same is not true for the pragmatics of a language. Let us recall that the pragmatics of a language answers the question “what is the purpose of this construct?” or “what use is a certain command?” It is clear, therefore, that the pragmatics of a programming language is not established once and for all during its definition (that is, the definition of its syntax and semantics). On the contrary, it evolves with the use of that language. The suggestions about programming style are part of pragmatics. For example, that jumps (`gotos`) should be avoided at all possible times. The choice of the most appropriate mode for passing parameters to a function is also a pragmatic question, as is the choice between bounded and unbounded iteration. Sometimes the definition of a language will suggest that a certain use of a construct is the “preferred one”, but it does not rule out other uses.

In a sense, the pragmatics of a programming language coincides with software engineering (the discipline which studies methods for the design and production of software). As such, it is not treated in this book. For many other aspects, on the other hand, clarifying the purpose and use of constructs is an essential part of the study of a programming language. In the following, we will often make pragmatical comments, usually without explicitly labeling them as such.

2.7 Implementation

The final level of programming language description will not be considered in this book in details. Implementing a language means writing a compiler for it, as well as implementing an abstract machine for the compiler’s object language; or to write an interpreter and implement the abstract machine in the language in which the interpreter is written. Alternatively, as happens in practice, a mix of both techniques is employed. Once more, this is not a book in which we can be exhaustive about these matters. But it would not be correct to present the constructions of programming languages without some reference to their implementation, and their cost. Even without specifying the set of constructions in an interpreter, for each construct, we should always ask: “How is it to be implemented?”, “At what cost?” The answer to these questions will also help us to understand the linguistic aspects (why a certain construct is formed in a certain way), as well as the pragmatic ones (how can a certain construct best be used).

2.8 Summary

The chapter introduced the fundamental techniques for the description and implementation of a programming language.

- The distinction between syntax, semantics, pragmatics and implementation. Each of these disciplines describes a crucial aspect of the language.
- Context-free grammars. A formal method essential for the definition of a language's syntax.
- Derivation trees and ambiguity. Derivation trees represent the logical structure of a string. The existence of a unique tree for every string permits a canonical interpretation to be assigned to it.
- Static semantics. Not all syntactic aspects of a language are describable using grammars: Static semantic checks are concerned with eliminating from legal programs those strings which, though correct as far as the grammar is concerned, do not respect additional contextual constraints.
- The organisation of a compiler.
- Structured operational semantics. A formal method for describing the semantics of programming languages based on transition systems.

2.9 Bibliographical Notes

The literature on the topics covered in this chapter is vast, even considering only introductory material. We limit ourselves to citing [6], the latest edition of a classic text on formal languages used by generations of students. For compiler-construction methods, we refer the reader to [3, 7]. An introduction to operational and denotational semantics is [4].

2.10 Exercises

1. Consider the grammar G'' , obtained from that in Fig. 2.6 by substituting the productions for the non-terminal symbol T with the following:

$$T \rightarrow A \mid E * T.$$

Show that G'' is ambiguous.

2. Give the obvious ambiguous grammar for the conditional command `if`, then, `else`.
3. Using the grammar fragment in Fig. 2.7 as a reference point, construct a derivation tree for the string

```
if (expression1) if (expression2) command1
    else command2
```

Assume that the following derivations exist:

$Expression \Rightarrow^* expression1,$

$Expression \Rightarrow^* expression2,$

$StatementWithoutTrailingSubstatement \Rightarrow^* command,$

StatementWithoutTrailingSubstatement $\Rightarrow^*$ command.

4. Define a grammar that will generate all the pairs of balanced curly brackets (braces).
5. Define a grammar that generates the language $\{a^n b^m \mid n, m \geq 1\}$ using only productions of the form $N \rightarrow tM$ or $N \rightarrow t$, where N and M are any non-terminal symbol and t is any terminal symbol. Try to give an intuitive explanation of why there exists no grammar with these characteristics which generates the language $\{a^n b^n \mid n \geq 1\}$.
6. Modify the rule of Fig. 2.12 so as to describe a right-to-left evaluation of arithmetic expressions.
7. Calculate the computation of the command

$$X := 1; \text{ while } \neg(X == 3) \text{ do } X := (X + 1)$$

starting with a chosen state.

8. In Example 2.5, the last transition has as its right member only a state (and not a pair composed of a command and a state). Is this always the case in a finite computation? (Hint: consider the command $X := 0; X := (X - 1)$, starting with any state whatsoever which includes X .)
9. State what the computation corresponding to the following command is:

$$X := 1; \quad X := (X - 1); \quad X := (X - 1); \quad X := 5$$

10. Considering Exercises 8 and 9, state a criterion which allows the division of finite computations into those which are correct and those which terminate because of an error.

References

1. C.W. Morris, Foundations of the theory of signs, in *Writings on the Theory of Signs* (Mouton, The Hague, 1938), pp. 17–74
2. J. Gosling, B. Joy, G. Steele, G. Bracha, *The Java Language Specification*, 3/E (Addison Wesley, 2005). The last specification available at the time of printing is that of Java SE 19, <https://www.docs.oracle.com/javase/specs/>. Accessed 14 Feb. 2023
3. A.V. Aho, M.S. Lam, R. Sethi, J.D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. (Pearson Education, 2006)
4. G. Winskel, *The Formal Semantics of Programming Languages* (MIT Press, 1993)
5. G.D. Plotkin, A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981. Repr. *J. Log. Alg. Program.* **60–61**, 17–139 (2004)
6. J.E. Hopcroft, R. Motwani, J.D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. (Pearson, 2006)
7. A.W. Appel, *Modern Compiler Implementation in Java*, 2nd ed. (Cambridge University Press, 2002). This text exists also for C and ML



In this chapter, we will not be concerned with programming languages but with the limits of the programs that we can write, asking whether there exist problems that no program can solve. A motivation for this research is the question that we asked at the end of Sect. 2.3: that is, is it possible to construct a static semantic analyser which can verify constraints imposed by the programming language's definition? We will soon discover, however, that the answer to the question is rather more general. It is a kind of absolute limit to what can (and cannot) be done with a computer. Although the material in this chapter can appear abstract, our treatment is wholly elementary.

3.1 The Halting Problem

In Sect. 2.3, we asked if there exists a static semantic analyser able to determine whether a program may generate a division by zero error during execution. Instead of tackling this problem, we will examine a larger problem, one that is also more interesting. We will ask whether there exists a static analyser able to discover whether a program, when provided with certain input data, will loop.

A static analyser can be viewed as a program used as a subprogram of a compiler. Let us, therefore, fix a programming language, $\mathcal{L}$, in which we will write programs. We assume $\mathcal{L}$ to be sufficiently expressive to be possible to write in $\mathcal{L}$ the interpreter for itself.¹ Given a program, P , and an input x , we write $P(x)$ to denote the result of computing P on x . However, P might not terminate on x because of a loop or of infinite recursion. Writing $P(x)$, therefore, we do not necessarily indicate a

¹ Although this requirement seems strange at first sight, this is true for any general-purpose programming language, including rudimentary ones like assembly languages, or theoretical ones like Turing machines (see the box on Sect. 3.4). We will discuss this requirement in Sect. 3.2.1.

computation that terminates. Without loss of generality, we can now reformulate the question as:

Does there exist a program, H , that, taking as input a program P (in the language $\mathcal{L}$) and its input data x , will terminate and print “yes” if $P(x)$ terminates, and terminate and print “no” if $P(x)$ loops?

We emphasise that H has two inputs. It must always terminate (we do not want a compiler that loops!) and it must work on every program, P , written in $\mathcal{L}$ and every input x . We can then assume, again without loss of generality, that H is written in the language $\mathcal{L}$.

We now show our hand. We want to show that there exists no program H with the behaviour we have just discussed. We will argue by contradiction, assuming we can have such an H , and from this we will derive a contradiction. The reasoning might seem a little contorted at first sight but requires no advanced knowledge. The argument, however, is subtle and elegant; the reader should read it more than once and be sure of understanding the critical role of self application.

1. Let us assume, by contradiction, that we have a program H with the properties stated above.
2. Using H , we can write a program K , with only one input (which will be another program), with the following specification:

The program K , given P as input, terminates printing “yes” if $H(P, P)$ prints “no”; it goes into a loop if $H(P, P)$ prints “yes”.

Writing the program K , given that H is available, is simple. We read the input P , call H as a subprogram, passing P as first and second parameter. We wait until H terminates (this will certainly happen, given H ’s specification). If $H(P, P)$ terminates and prints “no” (we can assume that we can intercept this printing and stop it from appearing on the output), K prints “yes”. If, on the other hand, $H(P, P)$ prints “yes”, then K (intercepts this printing and) goes into an infinite loop programmed for the purpose.

If we recall the specification of H , we can summarise the semantics of K as:

$$K(P) = \begin{cases} \text{“yes”} & \text{if } P(P) \text{ does not terminate,} \\ \text{does not terminate} & \text{if } P(P) \text{ does terminate.} \end{cases} \quad (3.1)$$

At first sight, the application of P to itself seems strange. But it is not unusual— P will receive as input a string representing the text of P .²

² If we had assumed that the input to P was a number, then pass to P the numeric value resulting from reading as a number the coding of the characters in the text of P .

3. Let us now execute K on its own text. That is, we focus on $K(K)$. What is its behaviour? If we substitute K for P in (3.1), we obtain:

$$K(P) = \begin{cases} \text{"yes"} & \text{if } K(K) \text{ does not terminate,} \\ \text{does not terminate} & \text{if } K(K) \text{ does terminate.} \end{cases} \quad (3.2)$$

4. Now let us observe that (3.2) is absurd. It says that $K(K)$ terminates (printing "yes") when $K(K)$ fails to terminate and that it does not terminate when $K(K)$ terminates.
5. Where does this contradiction come from? Not from K itself, which, as we have seen, is a simple program which uses H . The contradiction arises from having to assume the existence of a program with the properties of H . Therefore, H cannot exist.

We have therefore proved that there exists no *decision procedure* (in the language $\mathcal{L}$) capable of checking whether any program in $\mathcal{L}$ terminates on an arbitrary input. We use here decision procedure in the technical sense of a program that: (i) works for arbitrary arguments; (ii) always terminates and (iii) determines (by responding "yes" or "no") those arguments which are solutions to the problem and those which are not.

This result, one of fundamental importance in computing, is called the *undecidability of the halting problem*. Many other interesting problems are undecidable in the same way. We will discuss some of them in the next section after first having discussed the characteristics of $\mathcal{L}$ from which the result derives. We can thus tackle the problem of the expressive power of programming languages.

3.2 Undecidable Problems

At first sight, the result we obtained in the last section can appear fairly limited. If we take a language different from $\mathcal{L}$, perhaps the program H can exist without generating a contradiction.

Upon reflection, though, we have not assumed much about $\mathcal{L}$. We have used $\mathcal{L}$ in an implicit way to define the program K , given H (that is, at Step 2 of the proof). To be able truly to write K , the following conditions must be satisfied:

1. There must be some form of conditional available in $\mathcal{L}$, so that the cases in the definition of K can be distinguished;
2. It must be possible to define functions which do not terminate in $\mathcal{L}$ (we must therefore have at our disposal some form of iteration or recursion).

At this level of detail, $\mathcal{L}$ is nothing special. What programming language does not provide these constructs? If a language provides these constructs, it can be used in place of $\mathcal{L}$ in the proof, showing that a program like H exists in *no* programming language worth its salt.

3.2.1 The Standard Model

There is another assumption we made on $\mathcal{L}$, that to be able to express its own interpreter. This is not a big requirement. Any programming language with a conditional statement and a general looping mechanism can be shown to be able to express its own interpreter, *provided* that we understand its programs as executed on an abstract machine with unbounded resources.³ This is a tacit assumption that is always made in the programming language community, and that we call the *standard model* for programming language execution.

It is clear that the standard model is a theoretical abstraction. No program will (can!) ever be executed with unbounded resources. Our computers are finite objects, and there are no truly infinite computations in nature. The standard model is to programming languages what movement without friction is to mechanics—a fundamental abstraction that makes possible affordable mathematics of the phenomena being described. Under the assumption of the finiteness of the executing machine (hence: no standard model), if a program does not terminate, it must do so by cycling through the same configurations. Therefore, non-termination could be *in principle* be decided by a brute force analysis of all possible computations, looking for a cycle through the same configurations. Observe that, since everything is finite, also input data come from a finite set. If, instead, we assume the standard model of execution, there could be infinite computations that never cycle on the same configurations. A simple example is the program

```
i = 0;  
while True {  
    i += 1;  
}
```

which does not terminate and never cycle on the same configurations, provided there are unbounded resources to store the increasingly large value for `i`. On a (real) finite machine, of course that program will terminate, by exhaustion of the available storing resources.

Once the standard model of execution is assumed, the undecidability of the halting problem is not a contingent fact, related to any particular programming language, nor is it the expression of our inability as programmers. On the contrary, it is a limitation that is in some way absolute, linked to the intuitive concepts of program (algorithm) and of programming language. It is a principle of nature, similar to the conservation of energy or to the principles of thermodynamics. In nature, there exist more problems and functions than there are programs that we can write. Among these problems there are also significant ones, the halting problem being one of them.

³ And with true unbounded arithmetic if the language expresses arithmetic data.

3.2.2 More Undecidable Problems

We will argue in Sect. 3.3 that the problems for which there exists a program for their solution—the *decidable* problems—are only a tiny part of the set of all possible problems. As with the halting problem, when we say that a problem is undecidable, we mean that there exists no program such that: (i) it accepts arbitrary arguments; (ii) it always terminates and (iii) it determines which arguments are solutions to the problem and which are not. We will stray too far if we begin a proof of the undecidability of any other important problems; the interested reader can consult any good text on the theory of computability. We can however list some undecidable problems without attempting to explain all the terms used or to undertake any proofs.

The following problems are undecidable:

- Determine whether a program computes a constant function;
- Determine whether two programs compute the same function;
- Determine whether a program terminates for every input;
- Determine whether a program diverges for every input;
- Determine whether a program, given an input, will produce an error during its execution;
- Determine whether a program will cause a type error during its execution (see the box on Sect. 8.3.2).

These undecidability results tell us that *completely general* software tools which automatically establish significant properties of programs are impossible.

3.3 Formalisms for Computability

To make the discussion of $\mathcal{L}$ more precise, we need to fix a programming language and show that the reasoning of the previous section applies to it. Historically, the first language in which the impossibility of writing program H was shown was the language of the Turing machine. The Turing machine language is a notation which is, at first sight, highly rudimentary. It was introduced in the 1930s by the mathematician Alan M. Turing. It is summarised in the *Turing Machines* box. It is at first sight surprising that such a rudimentary formalism (there is no predefined arithmetic; everything turns on the positioning of a finite number of symbols on a tape) could be good enough to express computations as sophisticated as those needed to write K on the basis of H . More surprising is the fact that there exists a Turing machine which acts as an interpreter for any other Turing machine. That is, a machine which, once given as input on its own tape the (appropriately coded) description of a machine M and an input x for it, executes the computation which M would do on x . This interpreter is, to all intents and purposes, a (very simple) computer like those that we know today (program and data in memory, and a fundamental cycle that interprets program instructions).

Turing Machines

A Turing machine is composed of an infinite tape, divided into cells, each of which stores a single symbol from a finite alphabet. A mobile head reads from and writes to the tape. At any time, the head is positioned over a single cell. The machine is controlled by a finite-state controller. At each step in the computation, the machine reads a symbol from the tape and, according to the state of the machine and symbol that it has read, the controller decides which symbol to substitute for it on the tape and if the head is to move to the left or to the right; the controller therefore enters another state (remember, there is a finite number of states). The controller of a Turing machine can be seen as its associated program.

A function is *computable in a language* $\mathcal{L}$ if there exists a program in $\mathcal{L}$ which computes it. More precisely, the (partial—see the box on Sect. 1.2.2) function $f : A \rightarrow B$ is computable if there exists a program P with the following properties: for each element $a \in A$, whenever P is executed on an input a , which we write $P(a)$, it terminates providing as an output $f(a)$ if $f(a)$ is defined; the computation $P(a)$ does not terminate if f is undefined at a .

We might expect that the functions computed by the Turing machines are fewer than those computed by a sophisticated, modern programming language. However, this is not the case. Since the 1930s, many formalisms have been introduced for expressing algorithms (programs). Amongst these, the General Recursive Functions of Church, Gödel, and Kleene (which make no reference to programming languages), the lambda-calculus (which we will examine in Sect. 11.6), and then all programming languages. Indeed, any of these formalisms and languages can be simulated by any other of them. That is, it is possible in each of these formalisms to write an interpreter for any of the others. From this, it follows that they are all equivalent in terms of the functions they compute. They all compute exactly the same functions as the Turing machine. This is frequently expressed by saying that *all programming languages are Turing complete* (or *Turing equivalent*). The undecidability results, then, can also be expressed by saying that there exist functions which cannot be computed (with a Turing machine, or, by the above, in any programming language).

If all languages are equivalent with respect to the functions they compute, it is clear that they are not equivalent as far as their flexibility of use, pragmatics, abstraction principles and so on, are concerned. And often this complex of properties is referred to as the *expressiveness* of a language. While all these computability formalisms provide a definite result about equivalence in terms of the functions that can be expressed, those same formalisms do not say anything about the expressiveness of languages. Indeed, expressiveness (as we defined it) is a subject that has not been treated in a sufficiently formal and theoretical way.

3.4 There Are More Functions Than Algorithms

In Sect. 3.1, we gave a specific example of a uncomputable function. We can give another proof that there exist functions which cannot be computed using a simple cardinality argument (though, unlike in Sect. 3.1, we will not produce a specific example). That is, we show that there are more functions than algorithms.

First of all, let us consider any formal system which allows us to express algorithms and which, for simplicity we can assume to be a programming language, $\mathcal{L}$. It is easy enough to see that the set of all possible (finite) programs that can be written in $\mathcal{L}$ is *denumerable*, that is, they can be put into one-one correspondence with the natural numbers (which we denote by $\mathbb{N}$). We can, in fact, first consider the set P_1 containing all programs of length one (which contain a single character), then the set P_2 containing all programs of length two, and so on. Every set, P_i , is finite and can be ordered, for example lexicographically (first all programs which begin with a , then all those which begin with b , and so on by succeeding characters). It is clear that by doing this, by taking into account the ordering produced by the subscript in the sets $P_1, P_2, \dots$, and then doing the same with the internal indices in each set, we can count (or enumerate) all possible programs and therefore put them into one-one correspondence with the natural numbers. In more formal terms, the cardinality of the set of all programs in $\mathcal{L}$ is equal to the cardinality of the natural numbers. Let us now consider the set $\mathcal{F}$ containing all the functions $\mathbb{N} \rightarrow \{0, 1\}$. An important theorem of Cantor states that this set is not denumerable—it has a cardinality strictly greater than that of $\mathbb{N}$. Given that every program computes a unique function, there are functions in $\mathcal{F}$ for which there exist no program in $\mathcal{L}$. Which is what we wanted to show.

Let us see a direct proof of the fact that $\mathcal{F}$ is not denumerable. Let us assume that $\mathcal{F}$ is denumerable, that is it is possible to write $\mathcal{F} = \{f_j\}_{j \in \mathbb{N}}$. Let us observe, first of all, that we can put $\mathcal{F}$ into one-one correspondence with the set $\mathcal{B}$ of all the infinite sequences of binary numbers. To each $f_j \in \mathcal{F}$, there corresponds the sequence $b_{j,1}, b_{j,2}, b_{j,3}, \dots$, where $b_{j,i} = f_j(i)$, for $i, j \in \mathbb{N}$. Therefore if $\mathcal{F}$ is denumerable, so too is the set $\mathcal{B}$. Since $\mathcal{B}$ is denumerable, we can enumerate its elements one after the other, listing, for each element (for every sequence) its binary digits. We can arrange such an enumeration in an infinite square matrix:

$$\begin{array}{cccc} b_{1,1}, & b_{1,2}, & b_{1,3}, & \dots \\ b_{2,1}, & b_{2,2}, & b_{2,3}, & \dots \\ b_{3,1}, & b_{3,2}, & b_{3,3}, & \dots \\ \vdots & & & \end{array}$$

where row j contains the sequence for the j th function. Writing $\bar{b}$ for the complement of the binary number b , let us now consider the sequence of binary numbers $\bar{b}_{1,1}, \bar{b}_{2,2}, \bar{b}_{3,3}, \dots$. This sequence (since it is an infinite sequence of binary numbers) is certainly an element of $\mathcal{B}$. However it does *not* appear in our matrix (and therefore does not appear in our enumeration) for the reason that each line of the matrix differs in at *least one* digit from the sequence. Indeed, take the generic

Church's Thesis

The proofs of equivalence of the various programming languages (and between the various computability formalisms) are genuine theorems. Given two languages $\mathcal{L}$ and $\mathcal{L}'$, write first in $\mathcal{L}$ the interpreter for $\mathcal{L}'$ and then write in $\mathcal{L}'$ the interpreter for $\mathcal{L}$. At this point $\mathcal{L}$ and $\mathcal{L}'$ are known to be equivalent. A proof of this type has been effectively given for all existing languages, so they are therefore provably equivalent. This argument leaves open the door to the possibility that sooner or later someone will find an *intuitively computable* function for which no program may be given in any existing programming language. All equivalence results proved over more than 80 years, however, amount to convincing evidence that this is impossible. In the mid-1930s, Alonzo Church proposed a principle (which since then has become known as Church's, or the Church-Turing, Thesis) that states this impossibility. We can formulate Church's Thesis as: every intuitively computable function is computed by a Turing machine.

In contrast to the equivalence results, Church's Thesis is not a theorem because it refers to a concept (that of intuitive computability) that is not amenable to formal reasoning. It is, rather, a philosophical principle that the computer science community assumes with considerable strength to be true, and that so far has not been discredited even by new computational paradigms, for example, quantum computing.

line j . On the diagonal, line j has the element $b_{j,j}$, while, by construction, the new sequence has the element $\bar{b}_{j,j}$ in position j .

We have therefore a contradiction: we had assumed that $\mathcal{F} = \{f_j\}_{j \in \mathbb{N}}$ was an enumeration of *all* functions (that is, of all sequences), while we have constructed a function (a sequence) which did not belong to the enumeration. Therefore the cardinality of $\mathcal{F}$ is strictly greater than that of $\mathbb{N}$.

It can be shown that $\mathcal{F}$ has the cardinality of the real numbers. This fact indicates that the set of programs (which is denumerable) is much smaller than that of all possible functions, and therefore of all possible problems.

3.5 Summary

The phenomenon of computation on which Computer Science is founded has its roots in the theory of computability which studies the formalisms in which one can express algorithms and their limits. The chapter has only presented the main result of this theory. This is a fact of the greatest importance, one that every computer scientist should know. The principal concepts which were introduced are:

- *Undecidability*: there exist many important properties of programs which cannot be determined in a mechanical fashion by an algorithm; amongst these is the halting problem.

- *Computability*: a function is computable when there exists a program which computes it. The undecidability of the halting problem assures us that there exist functions which are not computable.
- *Partiality*: the functions expressed by a program can be undefined on some arguments, corresponding to those data for which the program will fail to terminate.
- *Turing Completeness*: every general-purpose programming language computes the same set of functions as those computed by a Turing Machine.

3.6 Bibliographical Notes

The original undecidability result was presented in the paper by Turing [1], which should be necessary reading for every computer scientist with an interest in theory. This chapter informally present just one tiny bit of computability theory. For more in-depth account, we recommend [2], which we cited in Chap. 2, and the classic [3] which after more than 50 years continues to be one of the most authoritative references.

3.7 Exercises

1. Proof that the restricted halting problem is undecidable. That is, determine whether a program terminates when it is applied to itself. (Suggestion: if the problem were decidable, the program which decides it would have to have the same property as program K , which can be derived by contradiction in the usual fashion.)
2. Show that the problem of verifying whether a program computes a constant function is undecidable. Hint: given a generic program P , consider the program Q_P , with a single input, specified as follows:

$$Q_P(y) = \begin{cases} 1 & \text{if } P(P) \text{ terminates,} \\ \text{does not terminate} & \text{otherwise.} \end{cases}$$

- (i) Write the program Q_P ;
- (ii) assume now that P is a program such that $P(P)$ terminates. What is the behaviour of Q_P , as y varies?
- (iii) what is, on other hand, the behaviour of Q_P , as y varies, if $P(P)$ does not terminate?
- (iv) from (ii) and (iii), it can be obtained that Q_P computes the constant function *one* if and only if $P(P)$ terminates;
- (v) if it were now decidable whether a program computes a constant function, the restricted halting problem would also be decidable, given that the transformation that, given P , constructs Q_P is completely general.

References

1. A. Turing, On computable numbers, with an application to the Entscheidungsproblem. Proc. London Math. Soc. **42**, 230–365 (1936). A Correction, ibidem **43**, 544–546 (1937)
2. J.E. Hopcroft, R. Motwani, J.D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. (Pearson, 2006)
3. H. Rogers, *Theory of Recursive Functions and Effective Computability* (McGraw Hill, reprinted by MIT Press, 1967)